

Day 27 — Capstone Part 3 — Sleep and Wake-Up Loop

Goal

Complete the wireless sensor node by adding a low-power sleep cycle. The node wakes, reads the sensor, sends BLE notifications, then enters System OFF to achieve sub- μA quiescent current between readings.

What you will learn

- How to use `CONFIG_PM=y` to reduce active sleep current
- How to force System OFF using `pm_state_force`
- How to configure a GPIO wake source for System OFF wake-up
- How to measure and compare current consumption across power states

Overview

Battery life is the key constraint for wireless sensor nodes. The strategy is to minimise the active window (wake $\rightarrow$ read $\rightarrow$ transmit $\rightarrow$ sleep) and spend as long as possible in the lowest available power state.

State	Typical current
Active + BLE TX	8–12 mA
k_sleep with PM	5–15 μA
System OFF	0.4–0.7 μA

A CR2032 coin cell (220 mAh) lasts hours at 4 mA, but over a year at 8 μA average.

Timed sleep (DK-friendly, no external RTC)

```
#include <zephyr/pm/pm.h>

/* At end of main(), after BLE notify */
LOG_INF("Sleeping 60 s...");
k_sleep(K_SECONDS(60));
/* On real hardware, enter System OFF here instead */
```

Enable tickless idle in `prj.conf`:

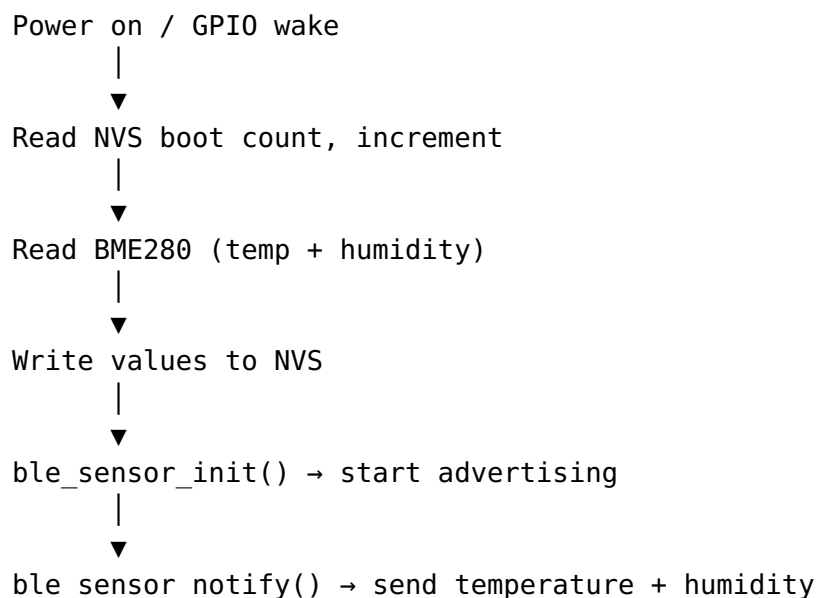
`CONFIG_PM=y`

System OFF (production pattern)

```
static void enter_system_off(void)
{
    LOG_INF("Entering System OFF");
    k_sleep(K_MSEC(100)); /* flush log backend */
    pm_state_force(0u, &(struct pm_state_info){PM_STATE_SOFT_OFF,
0, 0});
    /* Configure GPIO wake source (button sw0 = P0.11 on nRF52840
DK) */
    const struct device *gpio =
DEVICE_DT_GET(DT_NODELABEL(gpio0));
    gpio_pin_interrupt_configure(gpio, 11, GPIO_INT_LEVEL_LOW);
    k_sleep(K_FOREVER); /* does not return */
}
```

Call `enter_system_off()` at the end of `main()` after the BLE notify window.

Complete boot sequence



↓
▼
`k_sleep(K_SECONDS(2))` ← window for central to connect
↓
▼
`enter_system_off()` ← sub-μA until GPIO or timer wake

Measuring current

Use a Nordic PPK2 or a bench supply with current measurement: 1. Build with `CONFIG_PM=n` — measure baseline active current (~4 mA). 2. Add `CONFIG_PM=y` and `k_sleep(K_SECONDS(60))` — measure average current during sleep. 3. Switch to System OFF — confirm sub-μA quiescent current.

Why this matters

The System OFF → GPIO wake pattern is used in virtually every commercial coin-cell IoT product. Once the active window is under 100 ms and the duty cycle is low (one reading per minute), a CR2032 can power the node for over a year without a recharge.

Practice tasks

1. Flash the complete capstone firmware (Days 25–27) and confirm the boot counter increments.
2. Connect with nRF Connect and subscribe to both characteristics — confirm live readings.
3. Enable `CONFIG_PM=y` and measure current during `k_sleep(K_SECONDS(60))`.
4. Switch to System OFF — wake with button SW1 — confirm boot count increments on each wake.
5. Calculate expected battery life: CR2032 capacity ÷ average current over one 60-second cycle.

Example folder

View the complete source code on GitHub: [src/capstone_wireless_sensor_node](#)

Next topic

You have now completed the full 27-day course. Advanced topics for further study: MCUboot OTA firmware updates, BLE GATT profiles (HRS, ESS), LwM2M cloud integration, and production PCB bring-up.